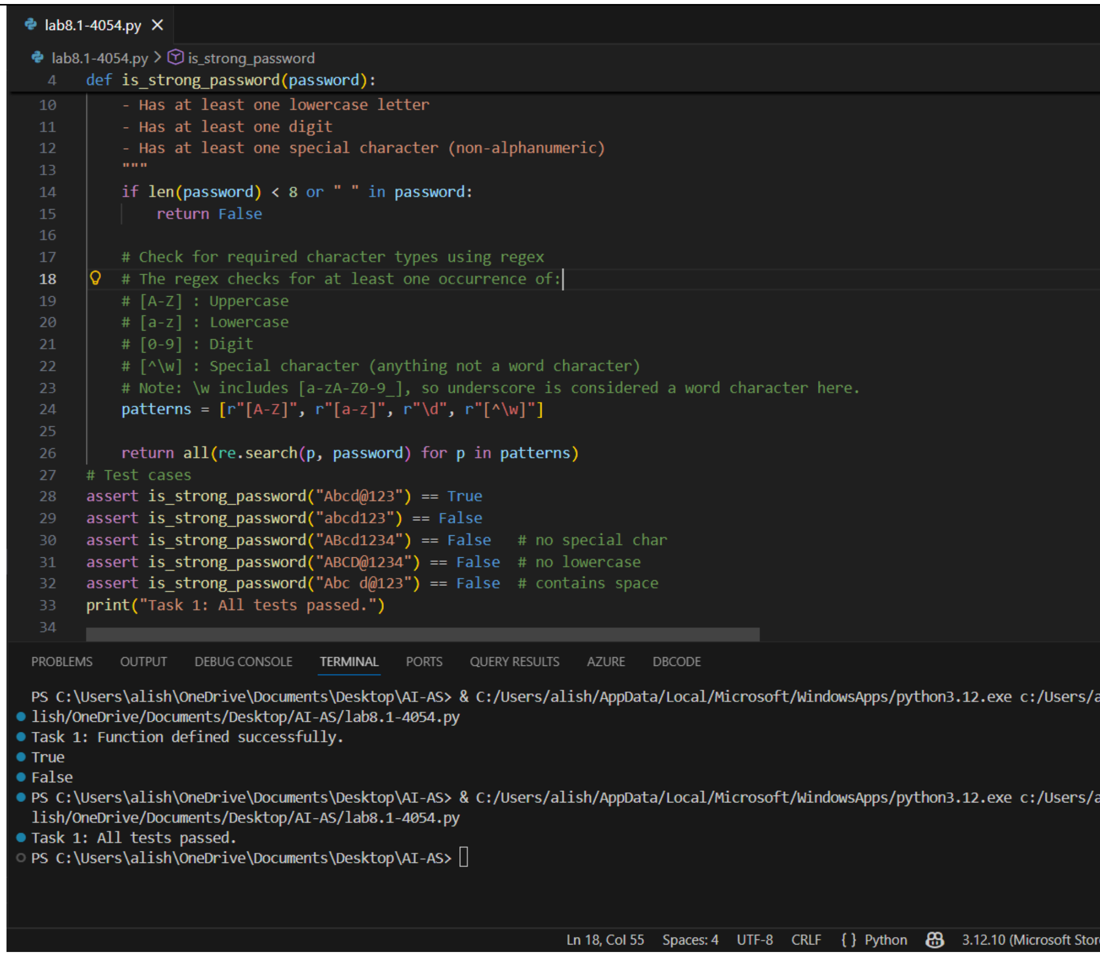
SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE			DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		**Assignment Type: Lab**	**Academic Year:** 2025-2026	
CourseCode	23CS002PC304	**Course Title**	AI Assisted Coding	
Year/Sem	III/II	**Regulation**	R23	
Date and Day of Assignment	**Week4 – Monday**	**Time(s)**	23CSBTB01 To 23CSBTB52	
Name	Aelishala Manoj	**Hall ticket No**	2303A54054	
Assignment Number: 8.1				

Q.No.	Question

1	Lab 8: Test-Driven Development with AI – Generating and Working with Test Cases
	Lab Objectives:
	• To introduce students to test-driven development (TDD) using AI code generation tools.
	• To enable the generation of test cases before writing code implementations.
	• To reinforce the importance of testing, validation, and error handling.
	• To encourage writing clean and reliable code based on AI-generated test expectations.
	Lab Outcomes (LOs):
	After completing this lab, students will be able to:
	• Use AI tools to write test cases for Python functions and classes.
	• Implement functions based on test cases in a test-first development style.
	• Use unittest or pytest to validate code correctness.
	• Analyze the completeness and coverage of AI-generated tests.
	• Compare AI-generated and manually written test cases for quality and logic
	
	Task Description #1 (Password Strength Validator – Apply AI in Security Context)
	• Task: Apply AI to generate at least 3 assert test cases for is_strong_password(password) and implement the validator function.
	• Requirements:
	  ○ Password must have at least 8 characters.
	  ○ Must include uppercase, lowercase, digit, and special character.
	  ○ Must not contain spaces.
	Example Assert Test Cases:
	assert is_strong_password("Abcd@123") == True
	assert is_strong_password("abcd123") == False
	assert is_strong_password("ABCD@1234") == True
	Expected Output #1:
	• Password validation logic passing all AI-generated test cases.



lab8.1-4054.py > is_strong_password

```python
def is_strong_password(password):
    - Has at least one lowercase letter
    - Has at least one digit
    - Has at least one special character (non-alphanumeric)
    """
    if len(password) < 8 or " " in password:
        return False

    # Check for required character types using regex
    # The regex checks for at least one occurrence of:
    # [A-Z] : Uppercase
    # [a-z] : Lowercase
    # [0-9] : Digit
    # [^\w] : Special character (anything not a word character)
    # Note: \w includes [a-zA-Z0-9_], so underscore is considered a word character here.
    patterns = [r"[A-Z]", r"[a-z]", r"\d", r"[^\w]"]

    return all(re.search(p, password) for p in patterns)
# Test cases
assert is_strong_password("Abcd@123") == True
assert is_strong_password("abcd123") == False
assert is_strong_password("ABcd1234") == False   # no special char
assert is_strong_password("ABCD@1234") == False  # no lowercase
assert is_strong_password("Abc d@123") == False  # contains space
print("Task 1: All tests passed.")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS AZURE DBCODE

```
PS C:\Users\alish\OneDrive\Documents\Desktop\AI-AS> & C:/Users/alish/AppData/Local/Microsoft/WindowsApps/python3.12.exe c:/Users/alish/OneDrive/Documents/Desktop/AI-AS/lab8.1-4054.py
Task 1: Function defined successfully.
True
False
PS C:\Users\alish\OneDrive\Documents\Desktop\AI-AS> & C:/Users/alish/AppData/Local/Microsoft/WindowsApps/python3.12.exe c:/Users/alish/OneDrive/Documents/Desktop/AI-AS/lab8.1-4054.py
Task 1: All tests passed.
PS C:\Users\alish\OneDrive\Documents\Desktop\AI-AS>
```

Ln 18, Col 55 Spaces: 4 UTF-8 CRLF { } Python 3.12.10 (Microsoft Store)

Task Description #2 (Number Classification with Loops – Apply AI for Edge Case Handling)

- Task: Use AI to generate at least 3 assert test cases for a classify_number(n) function. Implement using loops.
- Requirements:
  - Classify numbers as Positive, Negative, or Zero.
  - Handle invalid inputs like strings and None.
  - Include boundary conditions (-1, 0, 1).

Example Assert Test Cases:

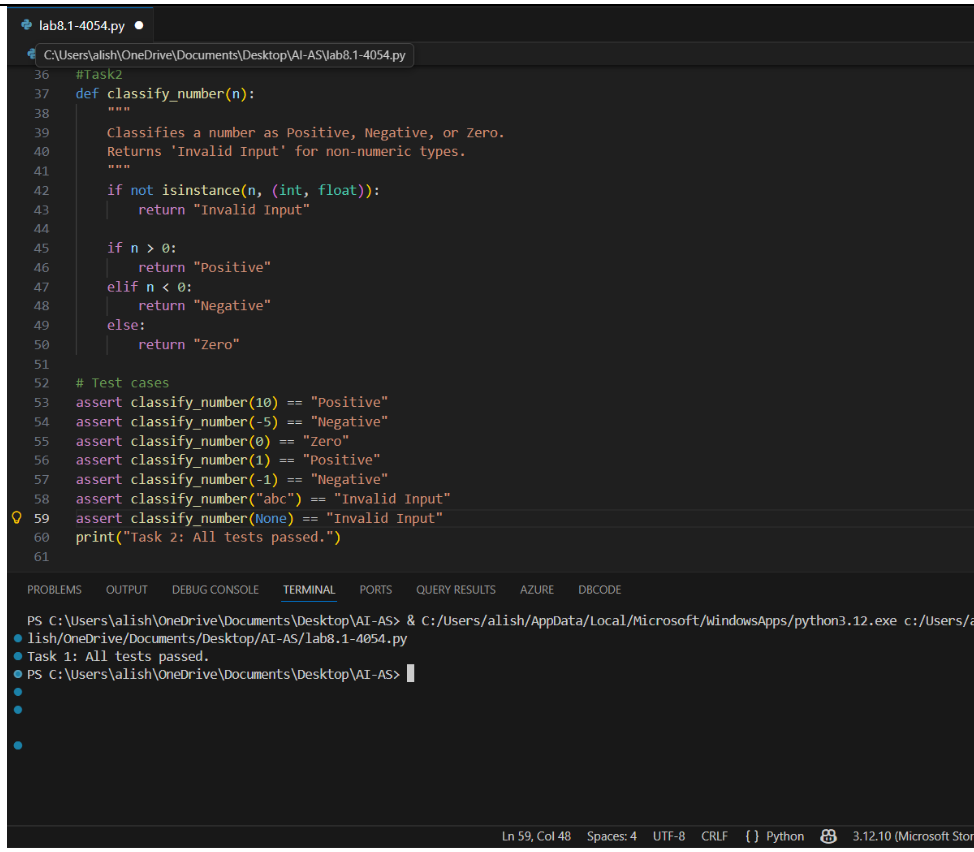
assert classify_number(10) == "Positive"

assert classify_number(-5) == "Negative"

assert classify_number(0) == "Zero"

Expected Output #2:

- Classification logic passing all assert tests.



Task Description #3 (Anagram Checker – Apply AI for String Analysis)

- Task: Use AI to generate at least 3 assert test cases for is_anagram(str1, str2) and implement the function.
- Requirements:
  - Ignore case, spaces, and punctuation.
  - Handle edge cases (empty strings, identical words).

Example Assert Test Cases:

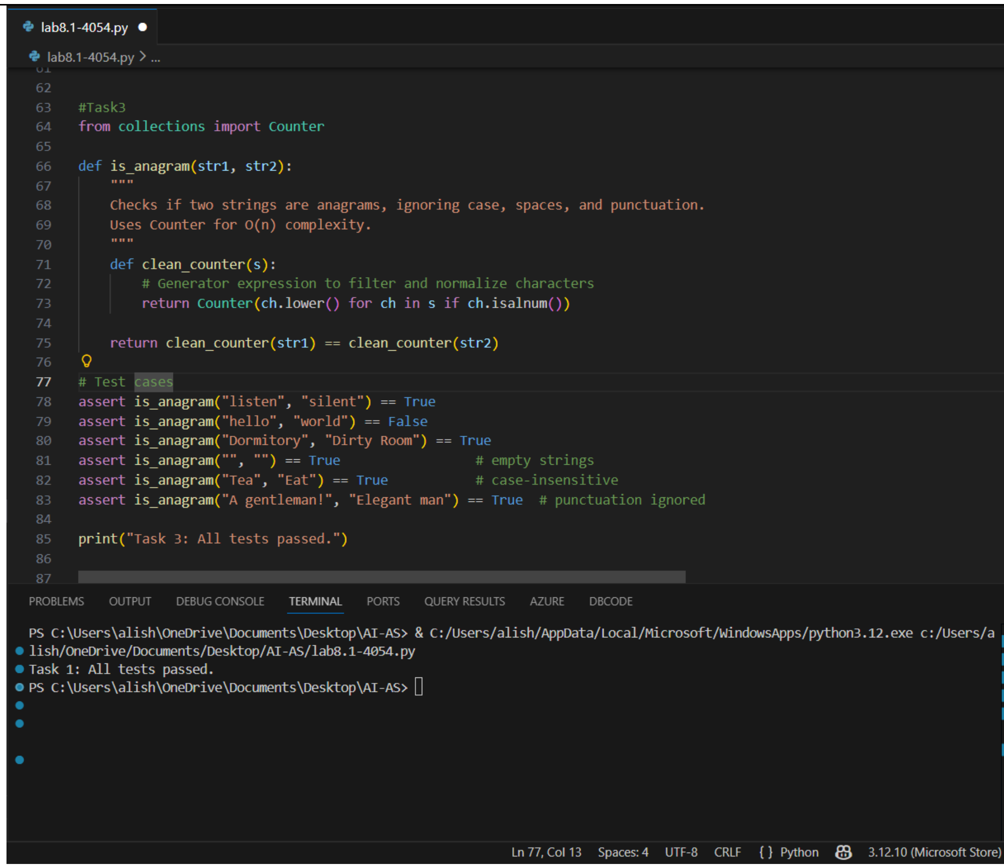
assert is_anagram("listen", "silent") == True

assert is_anagram("hello", "world") == False

assert is_anagram("Dormitory", "Dirty Room") == True

Expected Output #3:

- Function correctly identifying anagrams and passing all AI-generated tests.



Task Description #4 (Inventory Class – Apply AI to Simulate Real-World Inventory System)

- Task: Ask AI to generate at least 3 assert-based tests for an Inventory class with stock management.
- Methods:
  - add_item(name, quantity)
  - remove_item(name, quantity)
  - get_stock(name)

Example Assert Test Cases:

inv = Inventory()

inv.add_item("Pen", 10)

assert inv.get_stock("Pen") == 10

inv.remove_item("Pen", 5)

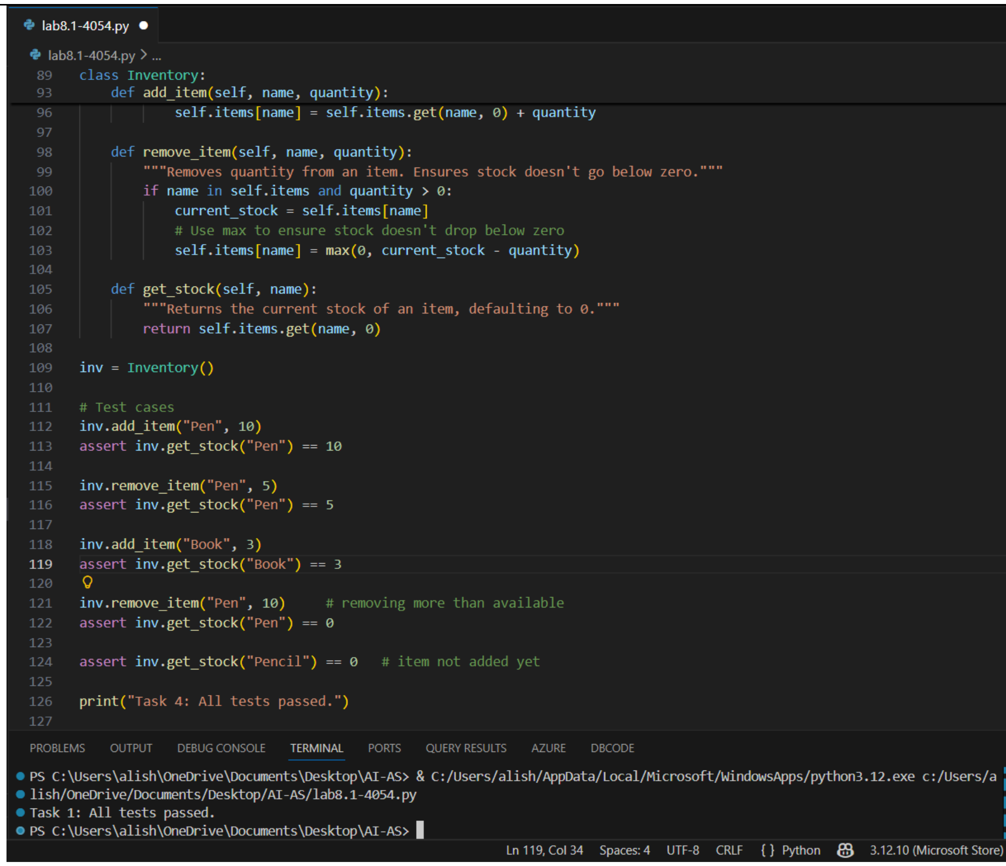
assert inv.get_stock("Pen") == 5

inv.add_item("Book", 3)

assert inv.get_stock("Book") == 3

Expected Output #4:

- Fully functional class passing all assertions.



```
lab8.1-4054.py ●
lab8.1-4054.py > ...
class Inventory:
    def add_item(self, name, quantity):
            self.items[name] = self.items.get(name, 0) + quantity

    def remove_item(self, name, quantity):
        """Removes quantity from an item. Ensures stock doesn't go below zero."""
        if name in self.items and quantity > 0:
            current_stock = self.items[name]
            # Use max to ensure stock doesn't drop below zero
            self.items[name] = max(0, current_stock - quantity)

    def get_stock(self, name):
        """Returns the current stock of an item, defaulting to 0."""
        return self.items.get(name, 0)

inv = Inventory()

# Test cases
inv.add_item("Pen", 10)
assert inv.get_stock("Pen") == 10

inv.remove_item("Pen", 5)
assert inv.get_stock("Pen") == 5

inv.add_item("Book", 3)
assert inv.get_stock("Book") == 3

inv.remove_item("Pen", 10)     # removing more than available
assert inv.get_stock("Pen") == 0

assert inv.get_stock("Pencil") == 0   # item not added yet

print("Task 4: All tests passed.")

PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   QUERY RESULTS   AZURE   DBCODE

PS C:\Users\alish\OneDrive\Documents\Desktop\AI-AS> & C:/Users/alish/AppData/Local/Microsoft/WindowsApps/python3.12.exe c:/Users/a
lish/OneDrive/Documents/Desktop/AI-AS/lab8.1-4054.py
Task 1: All tests passed.
PS C:\Users\alish\OneDrive\Documents\Desktop\AI-AS>

Ln 119, Col 34   Spaces: 4   UTF-8   CRLF   { } Python   3.12.10 (Microsoft Store)
```

Task Description #5 (Date Validation & Formatting – Apply AI for Data Validation)

- Task: Use AI to generate at least 3 assert test cases for validate_and_format_date(date_str) to check and convert dates.
- Requirements:
    - Validate "MM/DD/YYYY" format.
    - Handle invalid dates.
    - Convert valid dates to "YYYY-MM-DD".

Example Assert Test Cases:

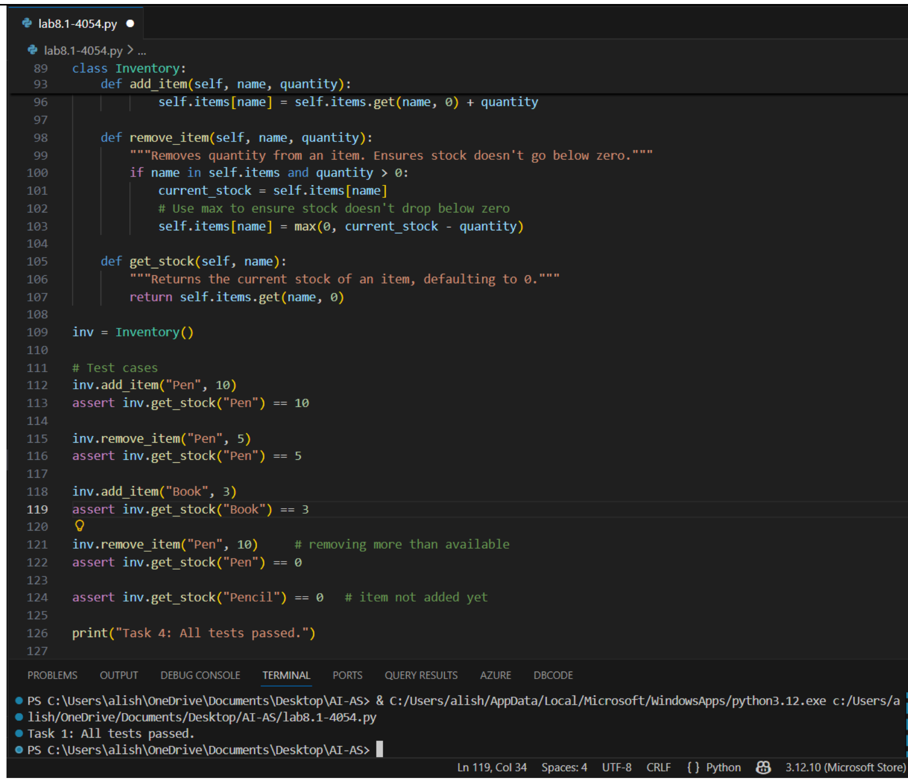
assert validate_and_format_date("10/15/2023") == "2023-10-15"

assert validate_and_format_date("02/30/2023") == "Invalid Date"

assert validate_and_format_date("01/01/2024") == "2024-01-01"

Expected Output #5:

- Function passes all AI-generated assertions and handles edge cases.



lab8.1-4054.py

lab8.1-4054.py > ...

```python
class Inventory:
    def add_item(self, name, quantity):
            self.items[name] = self.items.get(name, 0) + quantity

    def remove_item(self, name, quantity):
        """Removes quantity from an item. Ensures stock doesn't go below zero."""
        if name in self.items and quantity > 0:
            current_stock = self.items[name]
            # Use max to ensure stock doesn't drop below zero
            self.items[name] = max(0, current_stock - quantity)

    def get_stock(self, name):
        """Returns the current stock of an item, defaulting to 0."""
        return self.items.get(name, 0)

inv = Inventory()

# Test cases
inv.add_item("Pen", 10)
assert inv.get_stock("Pen") == 10

inv.remove_item("Pen", 5)
assert inv.get_stock("Pen") == 5

inv.add_item("Book", 3)
assert inv.get_stock("Book") == 3

inv.remove_item("Pen", 10)     # removing more than available
assert inv.get_stock("Pen") == 0

assert inv.get_stock("Pencil") == 0   # item not added yet

print("Task 4: All tests passed.")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS AZURE DBCODE

```
PS C:\Users\alish\OneDrive\Documents\Desktop\AI-AS> & C:/Users/alish/AppData/Local/Microsoft/WindowsApps/python3.12.exe c:/Users/a
lish/OneDrive/Documents/Desktop/AI-AS/lab8.1-4054.py
Task 1: All tests passed.
PS C:\Users\alish\OneDrive\Documents\Desktop\AI-AS>
```

Ln 119, Col 34 Spaces: 4 UTF-8 CRLF { } Python 3.12.10 (Microsoft Store)